ASSIGNMENT A03 / INDIVIDUAL CONTRIBUTION

Neural Network Safari

Data Engineering & Preprocessing

Team Member Cherluk

Role Data Engineering & Preprocessing Lead

1. Introduction & Objectives

My primary responsibility for the Neural Network Safari project was establishing a robust data pipeline to feed normalized image tensors into our modeling framework.

In the context of our safari metaphor, data engineering acts as the *habitat and nourishment* layer. It ensures that diverse data structures, ranging from standard tabular inputs to multidimensional visual inputs such as our Leopard CNN and Ostrich FNN datasets, are properly ingested, cleansed, transformed, and batched.

> **Core objective**
> Establish a reproducible, validated data pipeline that delivers correctly formatted tensors to downstream neural network architectures while reducing inconsistencies between preprocessing and model training.

2. Technical Execution & Methodology

To support our architecture zoo, I engineered preprocessing pipelines focused on four components.

01 Dataset Ingestion & Standardization

Handled dataset downloading and loading through PyTorch `torchvision`, ensuring reproducible training and testing splits. Standardized the input workflow so that the model-training stage could consume consistent datasets.

02 Tensor Transformations

Implemented image-normalization routines to scale pixel intensities to the range $[-1, 1]$. This transformation provides a consistent input scale and can support more stable optimization during backpropagation.

Normalization principle
For an input pixel $x \in [0, 1]$, the transformation

$$x_{\text{norm}} = 2x - 1$$

maps the original intensity interval to $[-1, 1]$.

03 Batch Configuration

Configured PyTorch `DataLoader` parameters, including appropriate batch sizes and pin-memory settings, to improve data-transfer efficiency and support GPU utilization in Google Colab.

04 Validation Routines

Added validation checks for tensor dimensions, data types, and label mappings before batches were passed to the downstream architecture modules. These checks helped identify data-format inconsistencies before they could interfere with training.

3. Challenges & Solutions

Challenge: Heterogeneous input dimensions
Different architectures require different input representations. A fully connected neural network (FNN) commonly consumes flattened feature vectors, whereas a convolutional neural network (CNN) preserves spatial structure in image tensors.

Solution: Modular tensor transformations
Created modular wrapper functions that reshape tensors according to the target architecture requested by teammates. This approach supported consistent data handoffs while allowing each model to receive the representation appropriate to its computational design.

4. Reflection

Building this data pipeline highlighted the critical dependency that neural networks have on clean, well-normalized inputs. Even a sophisticated architecture can perform poorly when preprocessing is inconsistent, features are incorrectly scaled, or data-loading bottlenecks prevent efficient training.

The project also reinforced the importance of designing interfaces between pipeline stages. By collaborating with teammates on tensor formatting and data handoffs, I helped establish a more consistent transition from data ingestion to model training.

Key insight
Model quality begins before the forward pass. Reliable preprocessing is not simply a preliminary task; it is a foundational part of the learning system. Correct tensor shapes, consistent normalization, reproducible data splits, and validated labels allow model architectures to learn from meaningful inputs rather than avoidable data inconsistencies.

CHERLUK ◇ DATA PIPELINE ◇ NEURAL NETWORK SAFARI